

Sprawozdanie z Laboratorium nr 2

Przedmiot: Integracja Systemów Informatycznych

Temat: Lokalna aplikacja Django – System Blogowy (Praca z gałęziami)

Data: 2026-04-15 **Student:** Aleksandr Vasiljev

1. Cel laboratorium

Celem laboratorium było stworzenie lokalnej aplikacji blogowej w Django oraz opanowanie pracy z systemem kontroli wersji Git w modelu Feature Branch Workflow. Dodatkowo celem było zapoznanie się z działaniem modeli, migracji, widoków oraz podstaw pracy z repozytorium GitHub.

2. Realizacja zadań

W trakcie laboratorium utworzono aplikację blogową w Django oraz rozwijano ją zgodnie z podejściem pracy na gałęziach (feature branch). Wszystkie zmiany były wykonywane w osobnej gałęzi **feature/blog-app**.

Zadanie 1: Praca na gałęziach (Git Workflow)

- **Opis działań:** Stworzono nową gałąź: **blog-app**.
- **Zastosowane komendy Git:**

```
git checkout -b feature/blog-app
```

Zadanie 2: Inicjalizacja aplikacji Django

- **Opis działań:** Utworzono nową aplikację Django o nazwie **blog** i zarejestrowano ją w pliku **settings.py**.
- **Zastosowane komendy Git:**

```
git checkout -b feature/blog-app
git add .
git commit -m "Add blog app to projects and settings"
```

Zrzut ekranu – settings

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'base',  
    'blog',  
]
```

Zadanie 3: Definicja modelu Post

- **Opis działań:** Stworzenie modelu `Post` z polami: `title`, `content`, `author`, `created_at`, `published_at`.

```
class Post(models.Model):  
    title = models.CharField(max_length=200)  
    content = models.TextField()  
    author = models.ForeignKey(User, on_delete=models.CASCADE)  
    created_at = models.DateTimeField(auto_now_add=True)  
    published_at = models.DateTimeField(default=timezone.now)  
  
    def __str__(self):  
        return self.title
```

- **Zastosowane komendy Git:**

```
git add .  
git commit -m "Define Post model"
```

Zadanie 4: Migracje i panel administracyjny

- **Opis działań:** Wykonanie migracji (`makemigrations`, `migrate`) i rejestracja w `admin.py`.

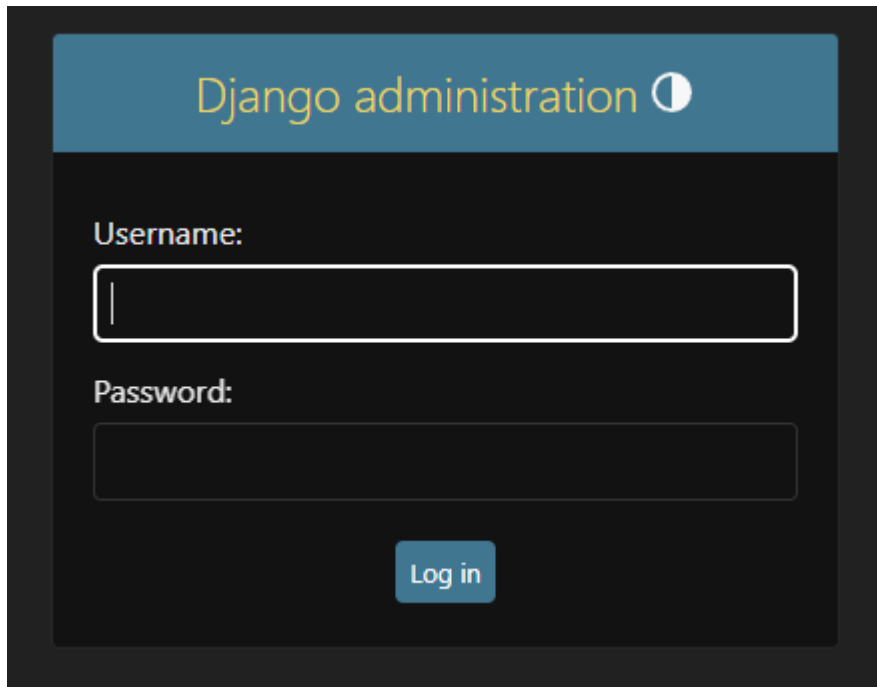
```
from django.contrib import admin  
from .models import Post  
  
admin.site.register(Post)
```

- **Zastosowane komendy Git:**

```
python manage.py makemigrations  
python manage.py migrate
```

```
git add .  
git commit -m "Initialize database and basic administrative access"
```

Zrzut ekranu – logowanie



Zadanie 5: Widoki i szablony

- **Opis działań:** Stworzenie widoku listy postów i szczegółów postu.

```
<h1>{{ post.title }}</h1>  
  
<p>{{ post.content }}</p>  
  
<a href="{% url 'post_list' %}">Powrót</a>
```

```
<h1>Lista postów</h1>  
  
<ul>  
  {% for post in posts %}  
    <li>  
      <a href="{% url 'post_detail' post.pk %}">  
        {{ post.title }}  
      </a>  
    </li>  
  {% endfor %}  
</ul>  
- **Zastosowane komendy Git:**
```

```
git add .  
git commit -m "Implement basic views and templates for blog posts"
```

Zrzut ekranu – Lista postów

Lista postów

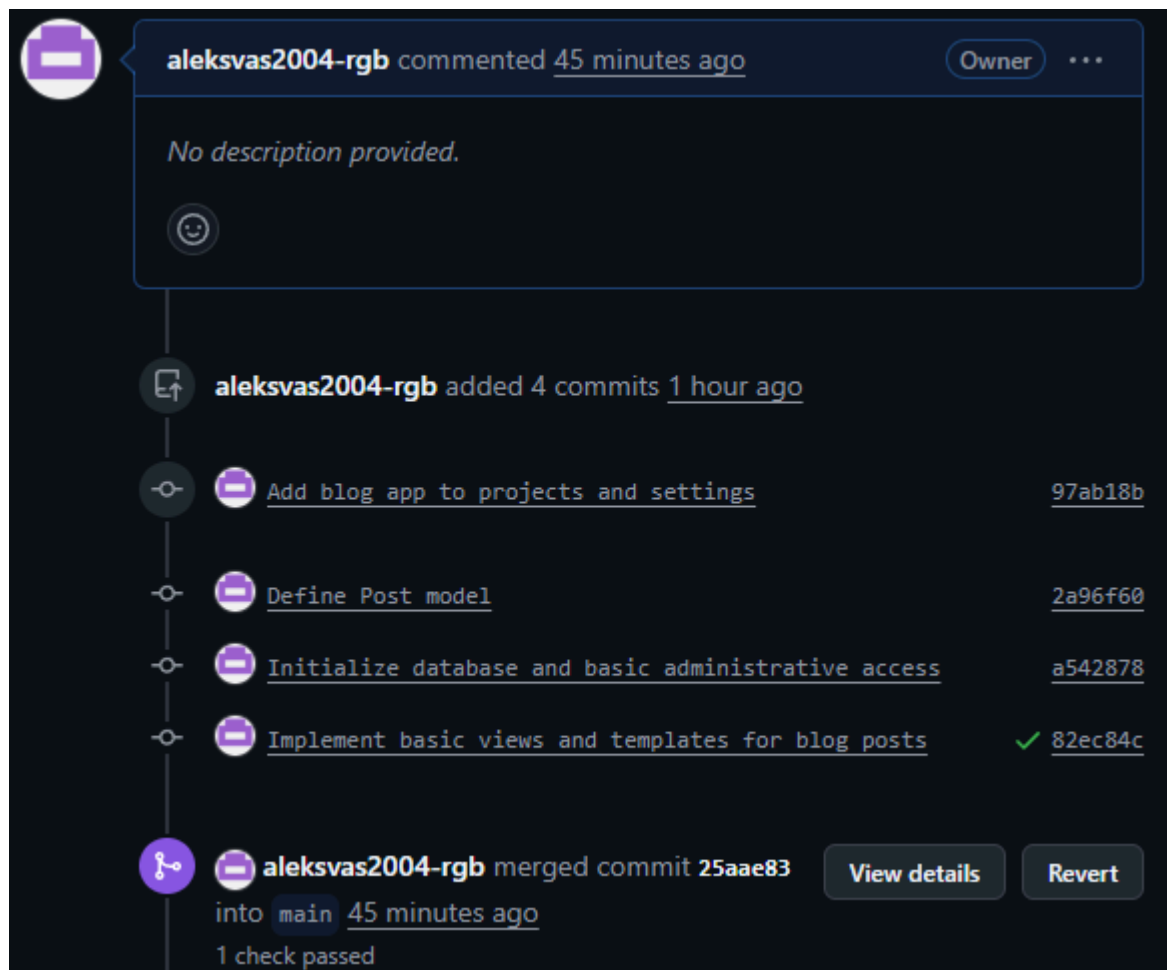
- [post1](#)
- [post2](#)

Zadanie 5: Zarządzanie zmianami i Merge

- **Opis działań:** Sprawdzenie statusu pracy oraz wypchnięcie do main z **Pull Request**.
- **Zastosowane komendy Git:**

```
git status  
git push origin feature/blog-app  
git log --oneline
```

Zrzut ekranu – Pull request



Wnioski

W trakcie realizacji laboratorium udało się stworzyć lokalną aplikację blogową w Django oraz poprawnie zintegrować ją z systemem kontroli wersji Git. Praca w modelu Feature Branch Workflow pozwoliła na lepszą organizację zmian w projekcie oraz ograniczenie ryzyka wprowadzania błędów do głównej gałęzi kodu.

Zastosowanie oddzielnych gałęzi dla poszczególnych funkcjonalności ułatwiło śledzenie historii zmian oraz zarządzanie rozwojem aplikacji. Dzięki wykorzystaniu mechanizmów Git (commit, push, merge, pull request) możliwe było kontrolowane wprowadzanie nowych funkcji do projektu.

Podczas pracy z Django zdobyto praktyczne doświadczenie w tworzeniu modeli, wykonywaniu migracji oraz rejestracji obiektów w panelu administracyjnym. Pozwoliło to lepiej zrozumieć sposób działania warstwy danych oraz administracji w frameworku.

Podsumowując, laboratorium pozwoliło na praktyczne utrwalenie wiedzy z zakresu Django oraz pracy z GitHub. Zastosowane narzędzia i metody są istotne w pracy zespołowej nad projektami programistycznymi i stanowią podstawę do dalszego rozwoju aplikacji webowych.